

**Name:** Gaurika Mittal

**Sap Id:** 590034892

**Batch:** 15

## **Experiment-19**

### **1. Algorithms**

#### **Operation 1 — Sphere: Total Surface Area and Volume**

**Function:** `sphere_area(double radius)` and `sphere_volume(double radius)`

1. Start.
2. Input radius  $r$ .
3. Check whether  $r > 0$ .
4. If not, display an error and input radius again.
5. Calculate total surface area:  $4 \times \text{PI} \times r \times r$ .
6. Calculate volume:  $(4/3) \times \text{PI} \times r \times r \times r$ .
7. Display total surface area and volume.
8. Stop.

#### **Operation 2 — Cylinder: Total Surface Area and Volume**

**Function:** `cylinder_area(double radius, double height)` and `cylinder_volume(double radius, double height)`

1. Start.
2. Input radius  $r$  and height  $h$ .
3. Validate that both values are positive.
4. Calculate total surface area:  $2 \times \text{PI} \times r \times (r + h)$ .
5. Calculate volume:  $\text{PI} \times r \times r \times h$ .
6. Display total surface area and volume.
7. Stop.

#### **Operation 3 — Cone: Total Surface Area and Volume**

**Function:** `cone_area(double radius, double height)` and `cone_volume(double radius, double height)`

1. Start.
2. Input radius  $r$  and height  $h$ .
3. Validate that both values are positive.
4. Calculate  $r^2 + h^2$ .

5. Find slant height using the Newton-Raphson square-root function:  $l = \text{sqrt}(r^2 + h^2)$ .
6. Calculate total surface area:  $\text{PI} \times r \times (r + l)$ .
7. Calculate volume:  $(\text{PI} \times r \times r \times h) / 3$ .
8. Display total surface area and volume.
9. Stop.

## Operation 4 — Cuboid: Total Surface Area and Volume

**Function:** `cuboid_area()` and `cuboid_volume()`

1. Start.
2. Input length  $l$ , breadth  $b$ , and height  $h$ .
3. Validate that all dimensions are positive.
4. Calculate total surface area:  $2 \times (l \times b + b \times h + h \times l)$ .
5. Calculate volume:  $l \times b \times h$ .
6. Display total surface area and volume.
7. Stop.

## Operation 5 — Base Area of Cylinder/Cone

**Function:** `circular_base_area(double radius)`

1. Start.
2. Input radius  $r$ .
3. Validate that radius is positive.
4. Calculate base area:  $\text{PI} \times r \times r$ .
5. Display base area.
6. Stop.

## Operation 6 — Base Area of Cuboid

**Function:** `cuboid_base_area(double length, double breadth)`

1. Start.
2. Input length  $l$  and breadth  $b$ .
3. Validate both values.
4. Calculate base area:  $l \times b$ .
5. Display base area.
6. Stop.

## Operation 7 — Mass of Solid

**Function:** `calculate_mass(double density)`

1. Start.
2. Input density of the solid.
3. Validate density.
4. Select the solid type.

5. Input and validate the required dimensions.
6. Calculate the volume of the selected solid.
7. Calculate mass using  $\text{Mass} = \text{Density} \times \text{Volume}$ .
8. Return and display mass.
9. Stop.

## Operation 8 — Weight of Solid

**Function:** `calculate_weight()`

1. Start.
2. Input density.
3. Select solid type and input its dimensions.
4. Calculate its volume.
5. Calculate mass:  $\text{Mass} = \text{Density} \times \text{Volume}$ .
6. Calculate weight:  $\text{Weight} = \text{Mass} \times \text{GRAVITY}$ .
7. Display weight.
8. Stop.

## Operation 9 — Gravitational Potential Energy

**Function:** `potential_energy(double elevation)`

1. Start.
2. Input elevation.
3. Validate elevation.
4. Input density.
5. Select the solid and input its dimensions.
6. Calculate its volume.
7. Calculate mass.
8. Calculate potential energy:  $\text{PE} = \text{Mass} \times \text{GRAVITY} \times \text{Elevation}$ .
9. Display potential energy.
10. Stop.

## Operation 10 — Pressure Exerted on Base

**Function:** `calculate_pressure(double density)`

1. Start.
2. Input and validate density.
3. Select cylinder, cone, or cuboid.
4. Input and validate the required dimensions.
5. Calculate volume.
6. Calculate mass:  $\text{Density} \times \text{Volume}$ .
7. Calculate weight:  $\text{Mass} \times \text{GRAVITY}$ .
8. Calculate the base area.
9. Calculate pressure:  $\text{Pressure} = \text{Weight} / \text{Base Area}$ .
10. Display pressure.
11. Stop.

## Operation 11 — Buoyant Force

**Function:** `buoyant_force(double fluid_density)`

1. Start.
2. Input fluid density.
3. Validate fluid density.
4. Select the solid.
5. Input its dimensions.
6. Calculate volume of the solid.
7. Since it is completely submerged, displaced fluid volume equals solid volume.
8. Calculate buoyant force:  $\text{Fluid Density} \times \text{Volume} \times \text{GRAVITY}$ .
9. Display buoyant force.
10. Stop.

## Operation 12 — Determine Buoyancy

**Function:** `determine_buoyancy(double solid_density, double fluid_density)`

1. Start.
2. Input solid density and fluid density.
3. Validate both values.
4. Find the absolute difference between the densities.
5. If the difference is less than or equal to `TOLERANCE`, return `SUSPENDED`.
6. Otherwise, if solid density is less than fluid density, return `FLOATS`.
7. Otherwise, return `SINKS`.
8. Display the result.
9. Stop.

## Square Root — Newton-Raphson Method

**Function:** `square_root(double number)`

1. Start.
2. Take the number whose square root is required.
3. Set an initial guess.
4. Calculate a new guess using:  $\text{new\_guess} = (\text{guess} + \text{number}/\text{guess}) / 2$ .
5. Find the absolute difference between new guess and old guess.
6. Replace old guess with new guess.
7. Repeat until the difference is less than or equal to `TOLERANCE`.
8. Return the calculated square root.
9. Stop.

## 2. Test-Case Table

| Test Case | Input                                                                | Expected Output                                                | Type     |
|-----------|----------------------------------------------------------------------|----------------------------------------------------------------|----------|
| <b>1</b>  | Choice = 1, Radius = 2                                               | Area = 50.2656,<br>Volume = 33.5104                            | Valid    |
| <b>2</b>  | Choice = 2, Radius = 2,<br>Height = 5                                | Area = 87.9648,<br>Volume = 62.8320                            | Valid    |
| <b>3</b>  | Choice = 3, Radius = 3,<br>Height = 4                                | Area = 75.3984,<br>Volume = 37.6992                            | Valid    |
| <b>4</b>  | Choice = 4, Length = 2,<br>Breadth = 3, Height = 4                   | Area = 52.0000,<br>Volume = 24.0000                            | Valid    |
| <b>5</b>  | Choice = 1, Radius = 0,<br>then Radius = 2                           | Shows error for 0,<br>then Area = 50.2656,<br>Volume = 33.5104 | Invalid  |
| <b>6</b>  | Choice = 7, Density = -10,<br>then 10, Solid type = 1,<br>Radius = 1 | Rejects -10, then<br>Mass = 41.8880                            | Invalid  |
| <b>7</b>  | Choice = 12, Solid density = 800,<br>Fluid density = 1000            | The solid floats.                                              | Valid    |
| <b>8</b>  | Choice = 12, Solid density = 1000,<br>Fluid density = 1000           | The solid remains suspended.                                   | Boundary |

### 3. solid.h

```
#ifndef SOLID_H
```

```
#define SOLID_H
```

```
#define PI 3.1416
```

```
#define GRAVITY 9.8
```

```
#define TOLERANCE 0.000001
```

```
#define SPHERE 1
```

```
#define CYLINDER 2
```

```
#define CONE 3
```

```
#define CUBOID 4
```

```
#define FLOATS -1
#define SUSPENDED 0
#define SINKS 1

double square_root(double number);

double sphere_area(double radius);
double sphere_volume(double radius);

double cylinder_area(double radius, double height);
double cylinder_volume(double radius, double height);

double cone_area(double radius, double height);
double cone_volume(double radius, double height);

double cuboid_area(double length, double breadth, double height);
double cuboid_volume(double length, double breadth, double height);

double circular_base_area(double radius);
double cuboid_base_area(double length, double breadth);

double calculate_mass(double density);
double calculate_weight(void);
double potential_energy(double elevation);
double calculate_pressure(double density);
double buoyant_force(double fluid_density);

int determine_buoyancy(double solid_density, double fluid_density);

#endif
```

## 4. math\_utils.c

```
#include "solid.h"

double square_root(double number)
{
    double guess, next_guess, difference;

    if (number <= 0.0)
        return 0.0;

    guess = number;

    if (guess < 1.0)
        guess = 1.0;

    do
    {
        next_guess = 0.5 * (guess + number / guess);

        difference = next_guess - guess;

        if (difference < 0.0)
            difference = -difference;

        guess = next_guess;

    } while (difference > TOLERANCE);

    return guess;
}
```

## 5. geometry.c

```
#include "solid.h"
```

```
double sphere_area(double radius)
```

```
{  
    return 4.0 * PI * radius * radius;  
}
```

```
double sphere_volume(double radius)
```

```
{  
    return (4.0 / 3.0) * PI * radius * radius * radius;  
}
```

```
double cylinder_area(double radius, double height)
```

```
{  
    return 2.0 * PI * radius * (radius + height);  
}
```

```
double cylinder_volume(double radius, double height)
```

```
{  
    return PI * radius * radius * height;  
}
```

```
double cone_area(double radius, double height)
```

```
{  
    double slant_height;  
  
    slant_height = square_root(radius * radius + height * height);  
  
    return PI * radius * (radius + slant_height);  
}
```



```
}
```

```
double cone_volume(double radius, double height)
```

```
{
```

```
    return (PI * radius * radius * height) / 3.0;
```

```
}
```

```
double cuboid_area(double length, double breadth, double height)
```

```
{
```

```
    return 2.0 * (length * breadth +
```

```
        breadth * height +
```

```
        height * length);
```

```
}
```

```
double cuboid_volume(double length, double breadth, double height)
```

```
{
```

```
    return length * breadth * height;
```

```
}
```

```
double circular_base_area(double radius)
```

```
{
```

```
    return PI * radius * radius;
```

```
}
```

```
double cuboid_base_area(double length, double breadth)
```

```
{
```

```
    return length * breadth;
```

```
}
```

## 6. physics.c

```
#include <stdio.h>
```

```
#include "solid.h"
```

```
static double read_positive(const char *prompt)
```

```
{
```

```
    double value;
```

```
    int status;
```

```
    int ch;
```

```
    while (1)
```

```
    {
```

```
        printf("%s", prompt);
```

```
        status = scanf("%lf", &value);
```

```
        if (status == 1 && value > 0.0)
```

```
            return value;
```

```
        printf("Invalid input. Enter a positive value.\n");
```

```
    if (status != 1)
```

```
    {
```

```
        while ((ch = getchar()) != '\n' && ch != EOF)
```

```
        {
```

```
        }
```

```
    }
```

```
}
```

```
static int read_solid_choice(int allow_sphere)
```

```
{
```

```
int choice, status, ch;
```

```
while (1)
```

```
{
```

```
    if (allow_sphere)
```

```
    {
```

```
        printf("1. Sphere\n");
```

```
        printf("2. Cylinder\n");
```

```
        printf("3. Cone\n");
```

```
        printf("4. Cuboid\n");
```

```
    }
```

```
    else
```

```
    {
```

```
        printf("2. Cylinder\n");
```

```
        printf("3. Cone\n");
```

```
        printf("4. Cuboid\n");
```

```
    }
```

```
    printf("Enter solid type: ");
```

```
    status = scanf("%d", &choice);
```

```
    if (status == 1 &&
```

```
        ((allow_sphere && choice >= SPHERE && choice <= CUBOID) ||
```

```
        (!allow_sphere && choice >= CYLINDER && choice <= CUBOID)))
```

```
    {
```

```
        return choice;
```

```
    }
```

```
    printf("Invalid solid choice. Try again.\n");
```

```

    if (status != 1)
    {
        while ((ch = getchar()) != '\n' && ch != EOF)
        {
        }
    }
}

```

```

static double obtain_volume(int solid)
{
    double radius, height, length, breadth;

```

```

    if (solid == SPHERE)
    {
        radius = read_positive("Enter radius: ");
        return sphere_volume(radius);
    }

```

```

    if (solid == CYLINDER)
    {
        radius = read_positive("Enter radius: ");
        height = read_positive("Enter height: ");

        return cylinder_volume(radius, height);
    }

```

```

    if (solid == CONE)
    {
        radius = read_positive("Enter radius: ");

```

```
height = read_positive("Enter height: ");
```

```
return cone_volume(radius, height);
```

```
}
```

```
length = read_positive("Enter length: ");
```

```
breadth = read_positive("Enter breadth: ");
```

```
height = read_positive("Enter height: ");
```

```
return cuboid_volume(length, breadth, height);
```

```
}
```

```
double calculate_mass(double density)
```

```
{
```

```
int solid;
```

```
double volume;
```

```
solid = read_solid_choice(1);
```

```
volume = obtain_volume(solid);
```

```
return density * volume;
```

```
}
```

```
double calculate_weight(void)
```

```
{
```

```
double density, mass;
```

```
density = read_positive("Enter solid density: ");
```

```
mass = calculate_mass(density);
```

```

    return mass * GRAVITY;
}

double potential_energy(double elevation)
{
    double density, mass;

    density = read_positive("Enter solid density: ");
    mass = calculate_mass(density);

    return mass * GRAVITY * elevation;
}

double calculate_pressure(double density)
{
    int solid;
    double radius, height;
    double length, breadth;
    double volume, base_area;

    solid = read_solid_choice(0);

    if (solid == CYLINDER)
    {
        radius = read_positive("Enter radius: ");
        height = read_positive("Enter height: ");

        volume = cylinder_volume(radius, height);
        base_area = circular_base_area(radius);
    }

```

```

else if (solid == CONE)
{
    radius = read_positive("Enter radius: ");
    height = read_positive("Enter height: ");

    volume = cone_volume(radius, height);
    base_area = circular_base_area(radius);
}
else
{
    length = read_positive("Enter length: ");
    breadth = read_positive("Enter breadth: ");
    height = read_positive("Enter height: ");

    volume = cuboid_volume(length, breadth, height);
    base_area = cuboid_base_area(length, breadth);
}

return (density * volume * GRAVITY) / base_area;
}

```

```

double buoyant_force(double fluid_density)
{
    int solid;
    double volume;

    solid = read_solid_choice(1);
    volume = obtain_volume(solid);

    return fluid_density * volume * GRAVITY;
}

```

```

}

int determine_buoyancy(double solid_density, double fluid_density)
{
    double difference;

    difference = solid_density - fluid_density;

    if (difference < 0.0)
        difference = -difference;

    if (difference <= TOLERANCE)
        return SUSPENDED;

    if (solid_density < fluid_density)
        return FLOATS;

    return SINKS;
}

```

## 7.main.c

```

#include <stdio.h>
#include "solid.h"

static double read_positive_main(const char *prompt)
{
    double value;
    int status, ch;

    while (1)
    {

```



```

printf("%s", prompt);

status = scanf("%lf", &value);

if (status == 1 && value > 0.0)
    return value;

printf("Invalid input. Enter a positive value.\n");

if (status != 1)
{
    while ((ch = getchar()) != '\n' && ch != EOF)
    {
    }
}
}

int main(void)
{
    int choice, result, status, ch;

    double r, h, l, b;

    double density, fluid_density;

    double solid_density, elevation;

    do
    {
        printf("\n--- SOLID GEOMETRY AND PHYSICS MENU ---\n");
        printf("1. Sphere total surface area and volume\n");
        printf("2. Cylinder total surface area and volume\n");
    }

```

```
printf("3. Cone total surface area and volume\n");
printf("4. Cuboid total surface area and volume\n");
printf("5. Circular base area\n");
printf("6. Cuboid base area\n");
printf("7. Mass of a solid\n");
printf("8. Weight of a solid\n");
printf("9. Gravitational potential energy\n");
printf("10. Pressure on base\n");
printf("11. Buoyant force\n");
printf("12. Buoyancy condition\n");
printf("0. Exit\n");
```

```
printf("Enter choice: ");
status = scanf("%d", &choice);
```

```
if (status != 1)
{
    printf("Invalid menu input.\n");

    while ((ch = getchar()) != '\n' && ch != EOF)
    {
    }

    choice = -1;
    continue;
}
```

```
switch (choice)
{
    case 1:
```

```
r = read_positive_main("Enter radius: ");
```

```
printf("Total surface area = %.4f\n",  
      sphere_area(r));
```

```
printf("Volume = %.4f\n",  
      sphere_volume(r));
```

```
break;
```

case 2:

```
r = read_positive_main("Enter radius: ");
```

```
h = read_positive_main("Enter height: ");
```

```
printf("Total surface area = %.4f\n",  
      cylinder_area(r, h));
```

```
printf("Volume = %.4f\n",  
      cylinder_volume(r, h));
```

```
break;
```

case 3:

```
r = read_positive_main("Enter radius: ");
```

```
h = read_positive_main("Enter height: ");
```

```
printf("Total surface area = %.4f\n",  
      cone_area(r, h));
```

```
printf("Volume = %.4f\n",  
      cone_volume(r, h));
```

```
break;
```

case 4:

```
l = read_positive_main("Enter length: ");  
b = read_positive_main("Enter breadth: ");  
h = read_positive_main("Enter height: ");
```

```
printf("Total surface area = %.4f\n",  
      cuboid_area(l, b, h));
```

```
printf("Volume = %.4f\n",  
      cuboid_volume(l, b, h));  
break;
```

case 5:

```
r = read_positive_main("Enter radius: ");
```

```
printf("Base area = %.4f\n",  
      circular_base_area(r));  
break;
```

case 6:

```
l = read_positive_main("Enter length: ");  
b = read_positive_main("Enter breadth: ");
```

```
printf("Base area = %.4f\n",  
      cuboid_base_area(l, b));  
break;
```

case 7:

```
density =
```

```
        read_positive_main("Enter solid density: ");

        printf("Mass = %.4f\n",
               calculate_mass(density));
        break;

case 8:
    printf("Weight = %.4f\n",
           calculate_weight());
    break;

case 9:
    elevation =
        read_positive_main("Enter elevation: ");

    printf("Potential energy = %.4f\n",
           potential_energy(elevation));
    break;

case 10:
    density =
        read_positive_main("Enter solid density: ");

    printf("Pressure = %.4f\n",
           calculate_pressure(density));
    break;

case 11:
    fluid_density =
        read_positive_main("Enter fluid density: ");
```

```
printf("Buoyant force = %.4f\n",
      buoyant_force(fluid_density));
break;

case 12:
    solid_density =
        read_positive_main("Enter solid density: ");

    fluid_density =
        read_positive_main("Enter fluid density: ");

    result =
        determine_buoyancy(solid_density,
                           fluid_density);

    if (result == FLOATS)
        printf("The solid floats.\n");

    else if (result == SUSPENDED)
        printf("The solid remains suspended.\n");

    else
        printf("The solid sinks.\n");

    break;

case 0:
    printf("Program terminated.\n");
    break;
```

default:

```
printf("Invalid menu choice. Enter 0 to 12.\n");
```

```
}
```

```
} while (choice != 0);
```

```
return 0;
```

```
}
```

## 8.Compilation

```
Session Actions Edit View Help
(kali㉿kali)-[~]
$ gcc main.c geometry.c physics.c math_utils.c -o solid_project
(kali㉿kali)-[~]
$ ./solid_project

— SOLID GEOMETRY AND PHYSICS MENU —
1. Sphere total surface area and volume
2. Cylinder total surface area and volume
3. Cone total surface area and volume
4. Cuboid total surface area and volume
5. Circular base area
6. Cuboid base area
7. Mass of a solid
8. Weight of a solid
9. Gravitational potential energy
10. Pressure on base
11. Buoyant force
12. Buoyancy condition
0. Exit
Enter choice: 1
Enter radius: 2
Total surface area = 50.2656
Volume = 33.5104
```

```
— SOLID GEOMETRY AND PHYSICS MENU —
1. Sphere total surface area and volume
2. Cylinder total surface area and volume
3. Cone total surface area and volume
4. Cuboid total surface area and volume
5. Circular base area
6. Cuboid base area
7. Mass of a solid
8. Weight of a solid
9. Gravitational potential energy
10. Pressure on base
11. Buoyant force
12. Buoyancy condition
0. Exit
Enter choice: 2
Enter radius: 2
Enter height: 5
Total surface area = 87.9648
Volume = 62.8320
```

```
— SOLID GEOMETRY AND PHYSICS MENU —
1. Sphere total surface area and volume
2. Cylinder total surface area and volume
3. Cone total surface area and volume
4. Cuboid total surface area and volume
5. Circular base area
6. Cuboid base area
7. Mass of a solid
8. Weight of a solid
9. Gravitational potential energy
10. Pressure on base
11. Buoyant force
12. Buoyancy condition
0. Exit
Enter choice: 3
Enter radius: 3
Enter height: 4
Total surface area = 75.3984
Volume = 37.6992
```

```
— SOLID GEOMETRY AND PHYSICS MENU —
1. Sphere total surface area and volume
2. Cylinder total surface area and volume
3. Cone total surface area and volume
4. Cuboid total surface area and volume
5. Circular base area
6. Cuboid base area
7. Mass of a solid
8. Weight of a solid
9. Gravitational potential energy
10. Pressure on base
11. Buoyant force
12. Buoyancy condition
0. Exit
Enter choice: 4
Enter length: 2
Enter breadth: 3
Enter height: 4
Total surface area = 52.0000
Volume = 24.0000
```

```
— SOLID GEOMETRY AND PHYSICS MENU —
1. Sphere total surface area and volume
2. Cylinder total surface area and volume
3. Cone total surface area and volume
4. Cuboid total surface area and volume
5. Circular base area
6. Cuboid base area
7. Mass of a solid
8. Weight of a solid
9. Gravitational potential energy
10. Pressure on base
11. Buoyant force
12. Buoyancy condition
0. Exit
Enter choice: 1
Enter radius: 0
Invalid input. Enter a positive value.
Enter radius: 2
Total surface area = 50.2656
Volume = 33.5104
```



```
— SOLID GEOMETRY AND PHYSICS MENU —
1. Sphere total surface area and volume
2. Cylinder total surface area and volume
3. Cone total surface area and volume
4. Cuboid total surface area and volume
5. Circular base area
6. Cuboid base area
7. Mass of a solid
8. Weight of a solid
9. Gravitational potential energy
10. Pressure on base
11. Buoyant force
12. Buoyancy condition
0. Exit
Enter choice: 7
Enter solid density: -10
Invalid input. Enter a positive value.
Enter solid density: 10
1. Sphere
2. Cylinder
3. Cone
4. Cuboid
Enter solid type: 1
Enter radius: 1
Mass = 41.8880
```

```
— SOLID GEOMETRY AND PHYSICS MENU —
1. Sphere total surface area and volume
2. Cylinder total surface area and volume
3. Cone total surface area and volume
4. Cuboid total surface area and volume
5. Circular base area
6. Cuboid base area
7. Mass of a solid
8. Weight of a solid
9. Gravitational potential energy
10. Pressure on base
11. Buoyant force
12. Buoyancy condition
0. Exit
Enter choice: 12
Enter solid density: 800
Enter fluid density: 1000
The solid floats.
```

```
— SOLID GEOMETRY AND PHYSICS MENU —
1. Sphere total surface area and volume
2. Cylinder total surface area and volume
3. Cone total surface area and volume
4. Cuboid total surface area and volume
5. Circular base area
6. Cuboid base area
7. Mass of a solid
8. Weight of a solid
9. Gravitational potential energy
10. Pressure on base
11. Buoyant force
12. Buoyancy condition
0. Exit
Enter choice: 12
Enter solid density: 1000
Enter fluid density: 1000
The solid remains suspended.
```

— SOLID GEOMETRY AND PHYSICS MENU —

1. Sphere total surface area and volume
2. Cylinder total surface area and volume
3. Cone total surface area and volume
4. Cuboid total surface area and volume
5. Circular base area
6. Cuboid base area
7. Mass of a solid
8. Weight of a solid
9. Gravitational potential energy
10. Pressure on base
11. Buoyant force
12. Buoyancy condition
0. Exit

Enter choice: 0

Program terminated.

